Education and Income
Micro-Project 1

ANA500: Python for Data Science
Douglas Bobeck
September 20, 2026

Python code and output

This project covers Acquire and Prepare. Pandas loads and organizes the Adult income data. NumPy checks numerical values, summarizes the data, and creates two fields for a later comparison.

Problem: Look at the relationship between education and income in the Adult dataset.

Hypothesis: Adults with at least a bachelor's degree are more likely to earn above $50,000 than adults with less education. More education may help people qualify for jobs with higher pay. That explanation would need to be tested.

Scope: The hypothesis remains untested. This assignment ends at Prepare. No group income-rate comparison, statistical model, significance test, or causal conclusion is included.

Tools used: Python 3.13.3, Pandas 3.0.6, and NumPy 2.5.3 for data preparation, with PyCharm as the development environment.

Repository: github.com/mascon0423/ANA500

Source: Becker, B., & Kohavi, R. (1996). *Adult* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5XW20

UCI describes an extract from the 1994 Census database. The local CSV values and row order match the 32,561-record training split. The separate test split is not included. Dataset license: CC BY 4.0.

1. Preparation decisions

Original data. Read the CSV as strings so missing markers can be handled consistently. Keep the original file unchanged and record its SHA-256 fingerprint. Each row is an adult record. The fnlwgt field is the final survey weight.

Missing values. Treat ? and blank cells as missing. Workclass has 1,836 missing values, occupation 1,843, and native-country 583. In total, 2,399 records are affected because some rows have more than one missing value. Education and income are complete. Keep unknown values missing instead of guessing replacements.

Repeated records. Keep the 24 records that repeat an earlier row. The file has no person ID, so identical values could represent different people. A later project could check whether keeping these records affects the results.

Types and validity. Convert six numeric columns to integers and nine text columns to categorical objects. Reject nonnumeric, fractional, nonfinite, or implausible numbers. Check income categories and the education-to-code mapping. Education-num is an ordered category code, not literal years of education.

NumPy objects. Use a two-dimensional array for numerical range checks and percentiles. Use np.where to derive education_group and income_gt_50k. Bachelors, Masters, Prof-school, and Doctorate form the group with at least a bachelor's degree.

Outliers and valid zeros. Inspect distributions without automatic clipping. Zero gains and losses are valid, and their quartiles are zero. An automatic IQR cap could erase meaningful positive values. Retain plausible observed extremes such as age 90 and 99 work hours.

Final dataset. Keep all 32,561 records and all 15 original fields. Add two fields for 17 columns. Save missing values as blank CSV cells. The education groups contain 8,067 and 24,494 records. Their income rates have not been compared.

These summaries count each row equally and describe the 1994 sample. A population estimate would need to consider survey weights. Cleaning decisions may need to change if later analysis finds other issues.

2. Python code

File: micro_project_1.py. Line numbers are for reference. Long lines wrap for printing; the runnable source remains unchanged.

Source SHA-256: f9d53b385d675e03f43256cacfc79b245197918421deddc20536cad0118c8652

```
  1  """ANA500 Micro-Project 1: education and income, through Acquire and Prepare.
  2
  3  Author: Douglas Bobeck
  4  Question: Is the share earning above $50,000 greater among adults with a
  5  bachelor's degree or higher than among adults with less education?
  6  Hypothesis: The bachelor's-or-higher group has the greater share.
  7  This script prepares that comparison. It does not test the hypothesis.
  8  Source: Becker & Kohavi (1996), Adult, https://doi.org/10.24432/C5XW20
  9  """
 10
 11  from pathlib import Path
 12  import hashlib
 13  import json
 14  import platform
 15
 16  import numpy as np
 17  import pandas as pd
 18
 19
 20  PROJECT = Path(__file__).resolve().parent
 21  INPUT = PROJECT / "data" / "adult_income.csv"
 22  OUTPUT = PROJECT / "deliverables"
 23  COLUMNS = (
 24      "age workclass fnlwgt education education-num marital-status occupation "
 25      "relationship race sex capital-gain capital-loss hours-per-week "
 26      "native-country income"
 27  ).split()
 28  NUMERIC = ["age", "fnlwgt", "education-num", "capital-gain",
 29             "capital-loss", "hours-per-week"]
 30  # education-num is an ordered education code, not literal years in school.
 31  EDUCATION = dict(zip([
 32      "Preschool", "1st-4th", "5th-6th", "7th-8th", "9th", "10th", "11th",
 33      "12th", "HS-grad", "Some-college", "Assoc-voc", "Assoc-acdm",
 34      "Bachelors", "Masters", "Prof-school", "Doctorate",
 35  ], range(1, 17)))
 36  DEGREES = ["Bachelors", "Masters", "Prof-school", "Doctorate"]
 37
 38
 39  def prepare(raw):
 40      """Return a prepared copy and audit evidence; never modify the input."""
 41      # 1. ACQUIRE: require the documented schema before transforming values.
 42      # Importing as strings makes missing markers and conversion rules explicit.
 43      if list(raw.columns) != COLUMNS or raw.empty:
 44          raise ValueError("Expected a nonempty Adult CSV with the 15 columns.")
 45      data = raw.astype("string").copy()
 46
 47      # 2. PREPARE: trim text and recognize both '?' and empty cells as missing.
 48      # Keep missing categories unknown rather than guessing a person's job.
 49      for column in COLUMNS:
 50          data[column] = data[column].str.strip().replace({"?": pd.NA, "": pd.NA})
 51      # The UCI test split may append a period to income labels. Normalize only
 52      # the two documented variants; unrelated or invalid labels must still fail.
 53      data["income"] = data["income"].replace({"<=50K.": "<=50K", ">50K.": ">50K"})
 54      missing = data.isna().sum()
 55      missing_rows = int(data.isna().any(axis=1).sum())
 56
 57      # Validate the variables required for the planned education comparison.
 58      if data[["education", "income"]].isna().any().any():
 59          raise ValueError("Education and income must be present for every row.")
```

```
    if not data["income"].isin(["<=50K", ">50K"]).all():
        raise ValueError("Income contains an undocumented category.")
    if not data["education"].isin(EDUCATION).all():
        raise ValueError("Education contains an undocumented category.")

    # Convert six quantitative fields to integers. Reject missing, infinite,
    # fractional, and unparseable values instead of silently truncating them.
    for column in NUMERIC:
        values = pd.to_numeric(data[column], errors="raise")
        array = values.to_numpy(dtype=float, na_value=np.nan)
        if not np.isfinite(array).all() or not (array % 1 == 0).all():
            raise ValueError(f"{column} must contain finite, nonmissing integers.")
        data[column] = values.astype("int64")

    # NumPy arrays allow one elementwise check across all numerical records.
    # These broad plausibility bounds are validation rules, not outlier caps.
    # The observed 90-year ages and 99-hour workweeks are retained, not erased.
    numeric_array = data[NUMERIC].to_numpy(dtype=float)
    lower = np.array([17, 1, 1, 0, 0, 1])
    upper = np.array([120, np.inf, 16, np.inf, np.inf, 168])
    if not ((numeric_array >= lower) & (numeric_array <= upper)).all():
        raise ValueError("A numeric value falls outside the plausibility bounds.")
    if not data["education"].map(EDUCATION).eq(data["education-num"]).all():
        raise ValueError("Education labels disagree with their numeric codes.")

    # Identical attribute profiles are not proof of duplicated people: this
    # dataset contains no person identifier. Count repeats and retain all rows.
    duplicates = int(data.duplicated().sum())
    for column in set(COLUMNS) - set(NUMERIC):
        data[column] = data[column].astype("category")

    # Create explicit variables for a future comparison with np.where.
    # No comparison of group income rates or model fitting occurs in Project 1.
    data["education_group"] = pd.Categorical(np.where(
        data["education"].isin(DEGREES), "Bachelor's or higher", "Below bachelor's"
    ))
    data["income_gt_50k"] = np.where(data["income"].eq(">50K"), 1, 0)

    # Explore distributions with NumPy percentiles and Pandas frequency tables.
    # Zero capital gain/loss values are valid. Do not automatically cap them
    # using IQR: both quartiles can be zero for these sparse monetary fields.
    percentiles = pd.DataFrame(
        np.percentile(numeric_array, [0, 25, 50, 75, 100], axis=0).T,
        index=NUMERIC, columns=["minimum", "Q1", "median", "Q3", "maximum"]
    )
    audit = {
        "input_rows": len(raw), "input_columns": len(raw.columns),
        "output_rows": len(data), "output_columns": len(data.columns),
        "rows_removed": len(raw) - len(data),
        "missing_by_column": {k: int(v) for k, v in missing.items()},
        "rows_with_missing": missing_rows,
        "duplicate_profiles_beyond_first": duplicates,
        "income_counts": data["income"].value_counts().to_dict(),
        "education_group_counts": data["education_group"].value_counts().to_dict(),
        "numeric_percentiles": percentiles.to_dict(orient="index"),
        "prepared_dtypes": data.dtypes.astype(str).to_dict(),
        "hypothesis_tested": False,
    }
    return data, audit


def main():
    # File paths are based on this script's folder.
    # Read the source once and hash its bytes to identify the exact input file.
    source_hash = hashlib.sha256(INPUT.read_bytes()).hexdigest()
    raw = pd.read_csv(INPUT, dtype="string", keep_default_na=False)
```

```
    data, audit = prepare(raw)
    audit["input_sha256"] = source_hash
    audit["versions"] = {"Python": platform.python_version(),
                         "NumPy": np.__version__, "Pandas": pd.__version__}

    # Record reproducible evidence. The generated CSV represents missing values
    # as empty fields; Pandas read_csv recognizes them as missing on reload.
    OUTPUT.mkdir(exist_ok=True)
    data.to_csv(OUTPUT / "adult_income_prepared.csv", index=False, na_rep="")
    (OUTPUT / "preparation_audit.json").write_text(
        json.dumps(audit, indent=2, allow_nan=False) + "\n", encoding="utf-8"
    )
    report = "\n".join([
        "ANA500 MICRO-PROJECT 1 | Douglas Bobeck",
        "Scope: Acquire and Prepare. Hypothesis remains untested.",
        f"Input: {len(raw):,} records x {len(raw.columns)} columns",
        f"Prepared: {len(data):,} records x {len(data.columns)} columns",
        f"Rows removed: {audit['rows_removed']}",
        f"Source SHA-256: {source_hash}",
        f"Versions: {audit['versions']}",
        "\nMISSING VALUES (retained as unknown)",
        pd.Series(audit["missing_by_column"]).to_string(),
        f"Rows with at least one missing value: {audit['rows_with_missing']:,}",
        f"Repeated profiles beyond first occurrence:
    {audit['duplicate_profiles_beyond_first']}",
        "Repeated profiles retained because no person identifier is available.",
        "\nNUMPY NUMERIC DISTRIBUTION SUMMARY (unweighted sample)",
        pd.DataFrame(audit["numeric_percentiles"]).T.to_string(),
        "\nINCOME CATEGORY COUNTS (unweighted sample)",
        data["income"].value_counts().to_string(),
        "\nEDUCATION GROUP COUNTS (no income comparison)",
        data["education_group"].value_counts().to_string(),
        "\nPREPARED DATA TYPES", data.dtypes.to_string(),
        "\nVALIDATION: schema, required values, integer fields, ranges,",
        "income labels, and education-code agreement all passed.",
        "Historical 1994 sample. fnlwgt retained as a survey weight.",
        "These summaries are not weighted population estimates.",
        "No hypothesis test, causal conclusion, model, or recommendation.",
    ]) + "\n"
    (OUTPUT / "preparation_output.txt").write_text(report, encoding="utf-8")
    print(report, end="")


if __name__ == "__main__":
    main()
```

3. Output

Output from micro_project_1.py. The SHA-256 fingerprint identifies the input dataset.

```
ANA500 MICRO-PROJECT 1 | Douglas Bobeck
Scope: Acquire and Prepare. Hypothesis remains untested.
Input: 32,561 records x 15 columns
Prepared: 32,561 records x 17 columns
Rows removed: 0
Source SHA-256: f2c62076f19504d99a38b22badf445a7f42530ade6b827acf78dd143fbce38bb
Versions: {'Python': '3.13.3', 'NumPy': '2.5.3', 'Pandas': '3.0.6'}

MISSING VALUES (retained as unknown)
age                 0
workclass        1836
fnlwgt              0
education           0
education-num       0
marital-status      0
occupation       1843
relationship        0
race                0
sex                 0
capital-gain        0
capital-loss        0
hours-per-week      0
native-country    583
income              0
Rows with at least one missing value: 2,399
Repeated profiles beyond first occurrence: 24
Repeated profiles retained because no person identifier is available.

NUMPY NUMERIC DISTRIBUTION SUMMARY (unweighted sample)
                minimum        Q1    median        Q3    maximum
age                17.0      28.0      37.0      48.0       90.0
fnlwgt          12285.0  117827.0  178356.0  237051.0  1484705.0
education-num       1.0       9.0      10.0      12.0       16.0
capital-gain        0.0       0.0       0.0       0.0    99999.0
capital-loss        0.0       0.0       0.0       0.0     4356.0
hours-per-week      1.0      40.0      40.0      45.0       99.0

INCOME CATEGORY COUNTS (unweighted sample)
income
<=50K    24720
>50K      7841

EDUCATION GROUP COUNTS (no income comparison)
education_group
Below bachelor's        24494
Bachelor's or higher     8067

PREPARED DATA TYPES
age                  int64
workclass         category
fnlwgt               int64
education         category
education-num        int64
marital-status    category
occupation        category
relationship      category
race              category
sex               category
capital-gain         int64
capital-loss         int64
hours-per-week       int64
```

```
native-country     category
income             category
education_group    category
income_gt_50k         int64

VALIDATION: schema, required values, integer fields, ranges,
income labels, and education-code agreement all passed.
Historical 1994 sample. fnlwgt retained as a survey weight.
These summaries are not weighted population estimates.
No hypothesis test, causal conclusion, model, or recommendation.
```

4. Verification and interpretation

The companion check_micro_project_1.py passed its checks for dataset dimensions, missing counts, income counts, derived fields, valid zeros, input preservation, and rejection of invalid input. It also checks transformations on an independent two-record example.

Check output: PASS: dataset counts, transformations, source preservation, and invalid inputs.

The prepared data keep every record. Missing workclass, occupation, and native-country values remain missing. Education and income are complete, and their labels and codes passed the checks. The data are ready for a later comparison, but the hypothesis has not been tested.

An analysis of occupation or workclass would need to decide how to handle the missing values in those fields. Later work could also check the effect of repeated records and use survey weights for population estimates.

Files produced by the script: adult_income_prepared.csv, preparation_audit.json, and preparation_output.txt. These files support the presentation's preparation evidence.

References

Becker, B., & Kohavi, R. (1996). *Adult* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5XW20

UCI dataset details: https://archive.ics.uci.edu/dataset/2/adult

Attribute definitions, split sizes, and survey weights: UCI adult.names documentation